SignalR Realtime

Contents

SignalR Realtime **1**
- Scope . . . 1
- Official requirements . . . 1
- Project policy . . . 1
- Checklist . . . 2
- References . . . 2
- Related . . . 2

SignalR Realtime

Scope

Hub design, authorisation, and event contracts for live voting, playlist editing and control delegation. Rationale: `brain/decisions/ADR-0004-realtime-signalr.md`. Concurrency design: `brain/architecture/realtime-architec`

Official requirements

- The subject requires real-time multi-user playlist editing and a live voting chain, and it explicitly grades race-condition handling.
- ASP.NET Core SignalR documented behaviour for hubs, groups, transports and authentication.

Project policy

Hubs are transport, nothing else

- A hub method validates its arguments, dispatches a CQRS command or query, and returns. No business rules, no data access, no branching on domain state inside a hub.
- Hub methods are `async Task` and accept the connection's `CancellationToken`.

Groups and events

Hub	Group key	Server → client events
`EventHub`	`event:{eventId}`	`TrackSuggested`, `VoteChanged`, `QueueReordered`, `NowPlayingChanged`
`PlaylistHub`	`playlist:{playlistId}`	`TrackAdded`, `TrackRemoved`, `TrackMoved`, `CollaboratorChanged`
`ControlHub`	`user:{userId}`	`ControlGranted`, `ControlRevoked`, `ControlTakenOver`

- Event names are `PascalCase` past-tense facts. Payloads are typed DTOs shared with the REST contract.
- Every payload carries the aggregate id and a monotonic `version` so a client can detect a missed message and resynchronise over REST.

Authorisation

- Hub connections authenticate with the same JWT as REST (`access_token` query parameter for the WebSocket handshake, which is the documented SignalR mechanism).
- Group membership is authorised **server-side on every join** against visibility and licence rules. A client asking to join `playlist:42` proves nothing.
- Re-check authorisation on reconnect. Permissions may have been revoked mid-session.

Ordering and consistency

- Broadcast **after** the database transaction commits. Never from inside the transaction.
- The database is the arbiter of truth; a broadcast is a notification, not a state transfer.
- Clients treat every event as a hint to reconcile, and must tolerate duplicates and reordering.

Operational

- Keep payloads small — these run over mobile networks.
- Configure explicit `KeepAliveInterval` and `ClientTimeoutInterval`; do not rely on defaults matching the mobile client's expectations.
- Multi-instance scale-out requires a backplane. Single instance today — see the Open ADR.

Checklist

- ☐ No business logic inside any hub method
- ☐ Group join authorised server-side, re-checked on reconnect
- ☐ Broadcast happens post-commit only
- ☐ Payloads versioned and small
- ☐ Reconnect path tested with the client dropping mid-edit

References

- ASP.NET Core SignalR — https://learn.microsoft.com/aspnet/core/signalr/introduction
- SignalR authentication — https://learn.microsoft.com/aspnet/core/signalr/authn-and-authz
- JavaScript client — https://learn.microsoft.com/aspnet/core/signalr/javascript-client

Related

`../clean-architecture-cqrs/STANDARD.md` · `../../20-api/rest-api-design/STANDARD.md` · `../../40-security/authentication-authorization/STANDARD.md`